

Introduction

Our robotic arm project was built within the Stanford AI4ALL Summer Program (7/19 - 7/31 of 2026). Specifically, it was created in the Robotics cohort with the help of educational resources of the AI4ALL organization and Stanford center for Human centered AI. The members of the developing team are: **Lily Montero, Leo Sekhri, Dimitrios Mavrakis, Alistair Smith** and **Yvonne Guo**. Special credits to our mentor Yash Khairnar and the program Counselors.

Problem

Our project provides a part of the solution to the 9th UN Sustainability Goal (Industry, Innovation and Infrastructure). The problem is that farmers who grow multiple type of fuits and vegetables, do not have the right machinery to sort them efficiently. Instead the labor has to be done with human workforce, which is expensive and time consuming. The money spent into these operation could be used for increasing the field sustainability and reducing the consumer cost.

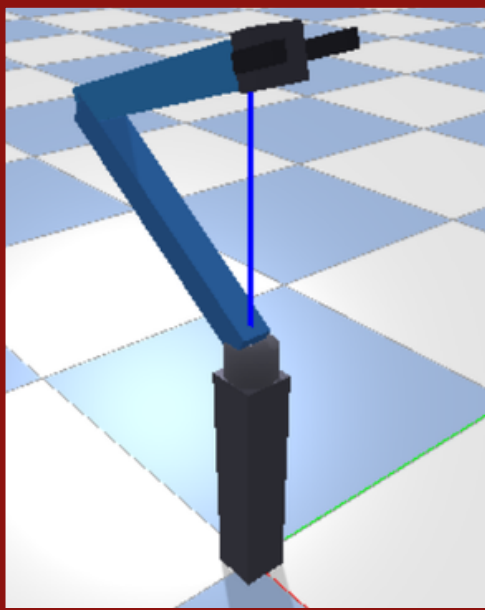


Solution

The solution is a robotics arm, that uses simplified artificial intelligence to sort fruits into different baskets. This machine is able to alalyze camera input, identify the fruits and vegetables in front of it and plan the most efficient path to sort them correctly. The specific implementation of AI in agriculture, can sort each fruit in 3.1s average, while it doesn't require any human observation.

Our Robotic Arm

Our robotic arm utilizes a robust but easy to build design, to sort the needed fruits. It has 2 degrees of freedom and contains 2 links and 2 joints. Additionally, it has a strong foundation that provides the required stability. The end effector is a precise claw that is able to apply a very good grip on the fruits.



Threshold

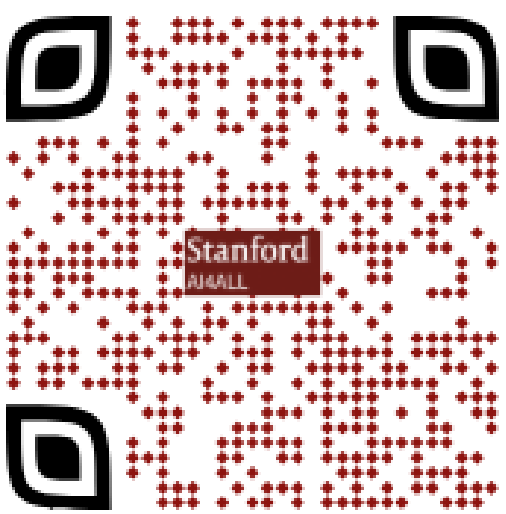
A threshold is a numerical value that is used as a benchmark to separate specific objects from a background. The brightness of every pixel is compared with this threshold. If the brightness of a pixel is greater than this threshold, it typically is turned white. If the brightness is less than the threshold, it is turned black. A threshold is a key part of our simulation and is a fundamental technique of computer vision.



Masking

Our simulation implements a common technique used in computer vision called masking. This is when a layer (called a mask) is applied on top of an original image. This mask changes the color values of the pixels in the images typically to either 0 (Black), or 255 (White), based on the threshold. This mask ultimately results in specific parts of the image to be isolated, and allows future algorithms to work with the isolated parts in an easier fashion.

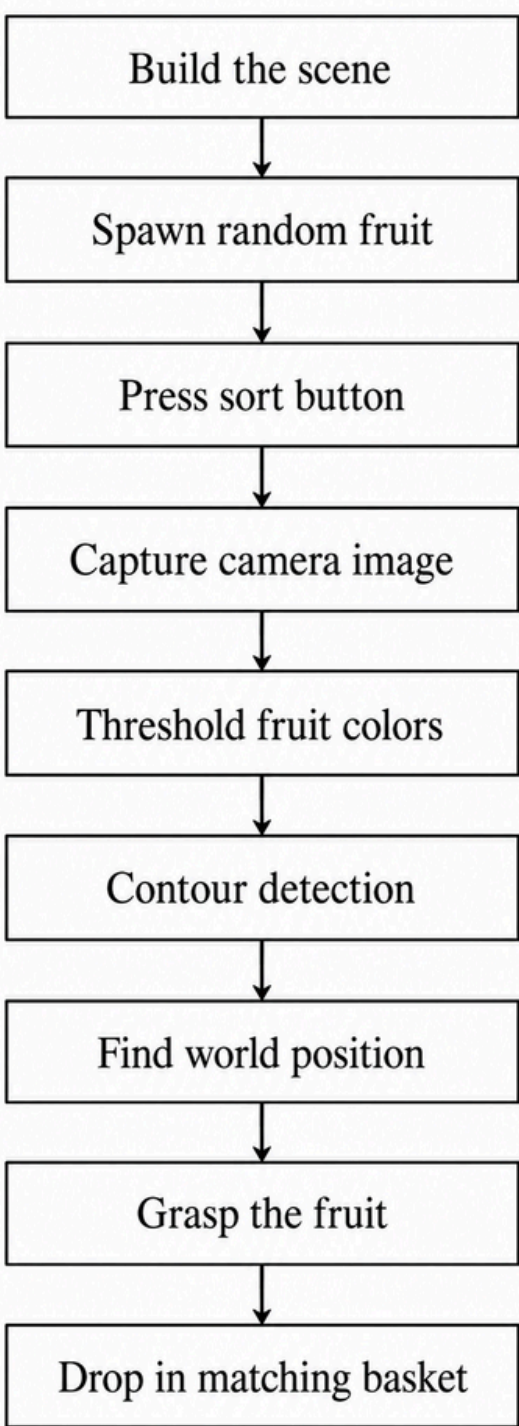
The simulation



The code



Flowchart

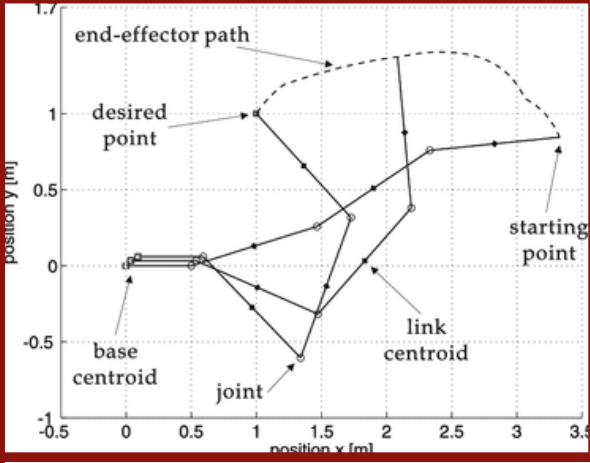
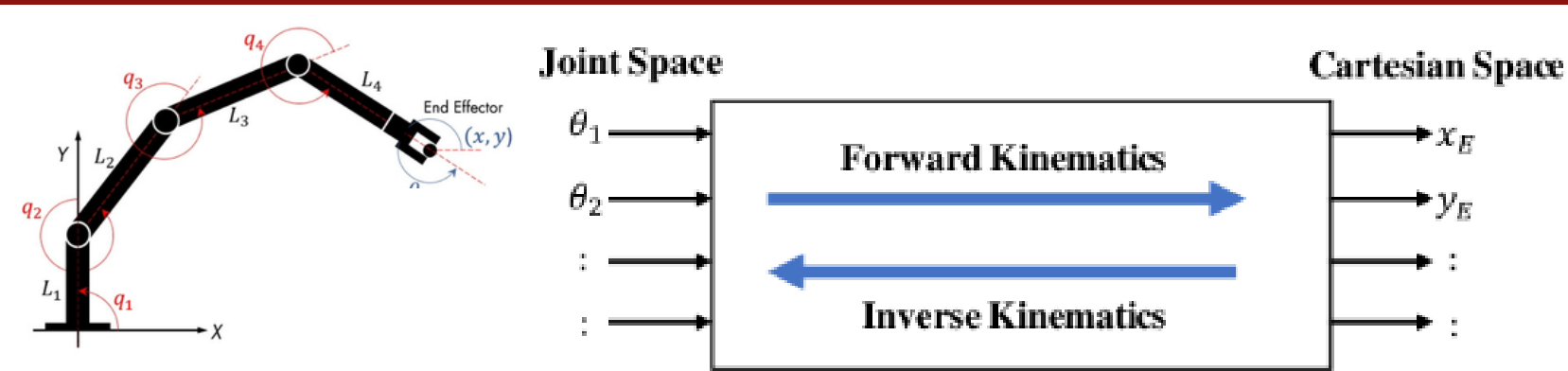


Contours

Contours are boundaries of distinct objects in an image. The original image is blurred, converted to HSV, and a mask is applied to separate the object from the background. After all of these steps, a contour detection algorithm is implemented to trace the object's boundary. This algorithm scans for where there is a large difference in the pixel values, and then draws a contour there, ultimately outlining the desired component in the image. From the contour, the robot can calculate the centroid (center), area, perimeter, and the shape of the desired object.

Inverse Kinematics

Inverse kinematics is the way our robotic arm knows how to move to a selected location. We use the Inverse Kinematics algorithym to calculate the angles of the joints that our robot need to have, in order to move to specific cordinates (x, y). It is commonly used in industrial robotic arms and its purpose is to accurately move the end effector to a disired point in the 3D simulation.



the Cartesian path planning ensures smooth and precise movements.

Path Planning

Our robotic arm uses Cartesian path planning, to plan the end effector's trajectory. To use it, we specify a series of points in the code's logic. Then Inverse Kinematics calculates what the arm has to do, in order to move the end effector to these points. Furthermore, the Cartesian path planning ensures smooth and precise movements.

Libraries



1. **pybullet** - Runs the physics simulation and 3D scene, creating the arm, fruit, table, and baskets, handling the camera, inverse kinematics, and grasping mechanism.



2. **numpy** - Handles the array math for camera matrices, coordinate calculations, and random fruit placement.



3. **cv2 (OpenCV)** - Processes the camera image to detect fruit by color, using HSV thresholding and contour detection to find each fruit's location.

